DesignGrad: Conceptual, Numerical and Visual Feedback for a Local CAD Coding Agent

Sinan Candan · Kaggle Gemma 4 Developer Agent Paper Track

Abstract. A mechanical design must be right in three ways: the idea must make sense, the numbers must work out, and the shape must come out as intended. We ask how much each kind of feedback helps a local model that designs parts as code. In **DesignGrad**, Gemma 4 26B-A4B runs offline on a laptop and plays every role: a designer writes a design document and CadQuery code, a CAD agent builds the part, and a critic's review becomes textual gradients that rewrite the design. We widen what the critic reads one step at a time: the design document alone (conceptual), measurements from analysis code the agent writes itself (numerical), and rendered views of the part (visual). Independent tests the agents never see score every design. Designs improve at each step, from 4.0 to 5.1 of seven requirements, and the three kinds of feedback do different jobs: text finds what the document leaves out, numbers say what is wrong, and a picture often says why. Self-written checks are kept honest by running them on a part with nothing in it.

1 Introduction

A coding agent that writes software can run the tests. A coding agent that writes a mechanical part has none: the program is a shape, and whether it is right depends on where the holes are, how much it weighs and whether it bends under load.

An engineer checks such a design in three ways. She reads it and asks whether the idea is sound. She calculates and measures. And she looks at the part. We give a local agent the same three kinds of feedback, one after another, and ask: **how do its designs improve as conceptual, numerical and visual feedback are added, and how should a local model be used for design?**

Everything is local and open: Gemma 4 26B-A4B [13] runs offline on a 32 GB laptop, where earlier physics-aware CAD agents relied on hosted models and fixed solvers [11, 12]. The agents never grade themselves; independent tests they cannot see decide every result.

2 The system

2.1 The loop

DesignGrad is a TextGrad-style loop [1] over two texts: a **design document** (concept, dimensions, how each requirement is met) and the **CadQuery program** that builds the part. One Gemma 4 model plays every role (Fig. 1). A designer reads the specification and writes both texts. A CAD agent runs the code and repairs it from the error messages until it builds one valid solid. A critic then reviews the design requirement by requirement. A gradient step turns the critique into concrete advice, first for the document and then for the code, and the designer rewrites both. This repeats up to four times.

The critic answers in three parts (Fig. 11). It first works through the requirements in notes, where it may calculate and change its mind, then gives one final verdict per requirement and a short list of problems, each naming the feature, the wrong value and the right one. Only verdicts and problems travel on: the designer receives a decision, not a debate.

After each round the loop scores the new design with what the agent can compute itself: whether the code builds, the share of requirements the critic believes are met, and the share its own measurements confirm. A design is kept only if this score strictly improves; otherwise the next round restarts from the best design.

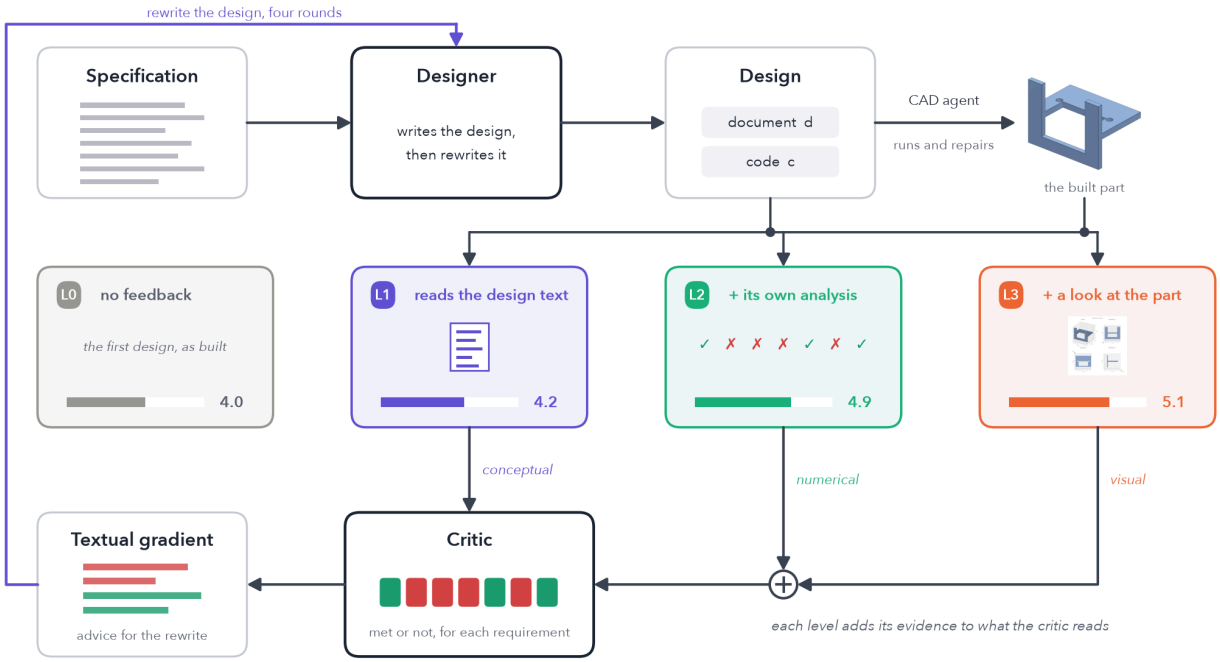


Fig. 1. The DesignGrad loop. The designer turns the specification into a design and the CAD agent builds it; the critic reviews it, and its critique becomes a textual gradient for the rewrite. Bars: requirements met (of 7).

2.2 The ladder

Every level writes a design and builds it. The levels differ in what the critic reads.

- **L0, no feedback.** The first design, as built.
- **L1, conceptual.** The critic reads only the design document. It is told not to trust the document's claims: it recomputes masses, stresses and positions from the stated dimensions, and asks for any missing dimension.
- **L2, numerical.** An analysis agent writes a program that measures every requirement on the built part, and the results join what the critic reads.
- **L3, visual.** A reviewer looks at four rendered views of the part beside the design document and lists what does not match. Its remarks join the rest.

Each level keeps everything below it, and all four start from the same first design.

2.3 The agent's own analysis, tested

Gemma writes the analysis program itself, for each problem, from the specification. Its prompt is the same for every problem and holds only a general reference sheet: library calls for loading and probing a part, a recipe for probing a hole, and textbook formulas such as those of a cantilever beam. Most of a program is geometric probing (is there material here, is that region empty); for the bracket it also applies the beam formulas to the section at the wall. A check that crashes goes back to the agent as an error, never to the critic as a failed requirement.

Self-written checks can be wrong in a way that is hard to see. A check for a hole that only tests whether a region is empty also passes when the plate around the hole is missing. The loop therefore tests the tests (Fig. 2). Each analysis program is run a second time on a control part, a one-millimetre cube with no features, and a reviewer is told which checks passed on it. A check that finds a hole or a wall on the cube measures nothing and is sent back. What survives is treated with care: a disputed "not met" still reaches the critic as a warning, a disputed "met" counts for nothing, and a requirement without a trusted measurement earns no credit.

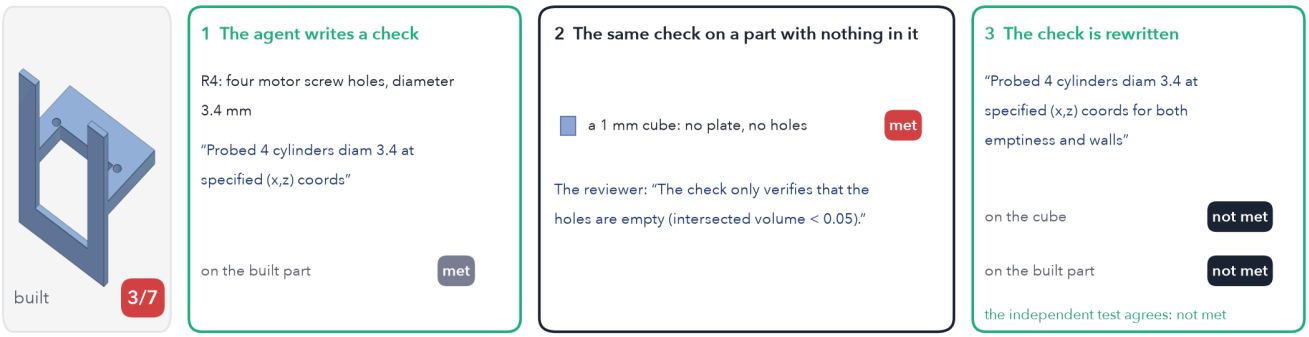


Fig. 2. Testing the tests (motor mount, L2). The agent's first check for the motor screw holes reports them present on a part whose face plate is missing, and on an empty cube as well. The reviewer sees the control result and sends the check back; the rewritten check is right.

3 Experiment

Problems. Three parts, each with seven numerical requirements (Fig. 4): a shelf bracket that must carry a load, an electronics enclosure base, and a motor mount. Each is run three times with different sampling seeds: nine runs per level. Four further parts, run twice each, test how far the picture carries (Section 4.5).

Independent tests. For each problem, geometric and analytic checks were written apart from the agents' tools and frozen before any run. A reference design meets all of them; a plain block fails most. "Requirements met" always refers to these tests.

Model. `gemma4:26b` in Ollama on an M1 Max laptop, temperature 0.3. The loop assumes that each agent finishes its own job, and on these three parts they did: of 217 agent tasks (building a part, writing an analysis), six ran out of steps.

4 Results

4.1 Step by step

Table 1. Final designs, nine runs per level.

Level	Requirements met (of 7)	Best design produced	Model calls	Minutes
L0 no feedback	4.0	4.0	2	1
L1 conceptual	4.2	5.0	14	4
L2 + numerical	4.9	5.1	39	14
L3 + visual	**5.1**	**5.7**	44	15

Each step up the ladder adds something (Table 1, Fig. 3). Reading the document alone changes little. Measurements lift the designs by almost one requirement, and a look at the part adds a little more: from L0 to L3 the designs gain 1.1 requirements (95% interval +0.4 to +1.9); five of nine runs improve and none gets worse. Each single step is within the noise of nine runs; the trend across the ladder is not.

The same order shows inside the loop. At L2 the design it holds improves a little in every round. At L3 it improves faster and then levels off. At L1 it wanders: a critic that cannot see the part sometimes approves a rewrite that built worse.

The parts differ (Fig. 3, bottom; Fig. 4). The enclosure is nearly right from the start and stays there. The bracket needs numbers. The motor mount needs to be looked at.

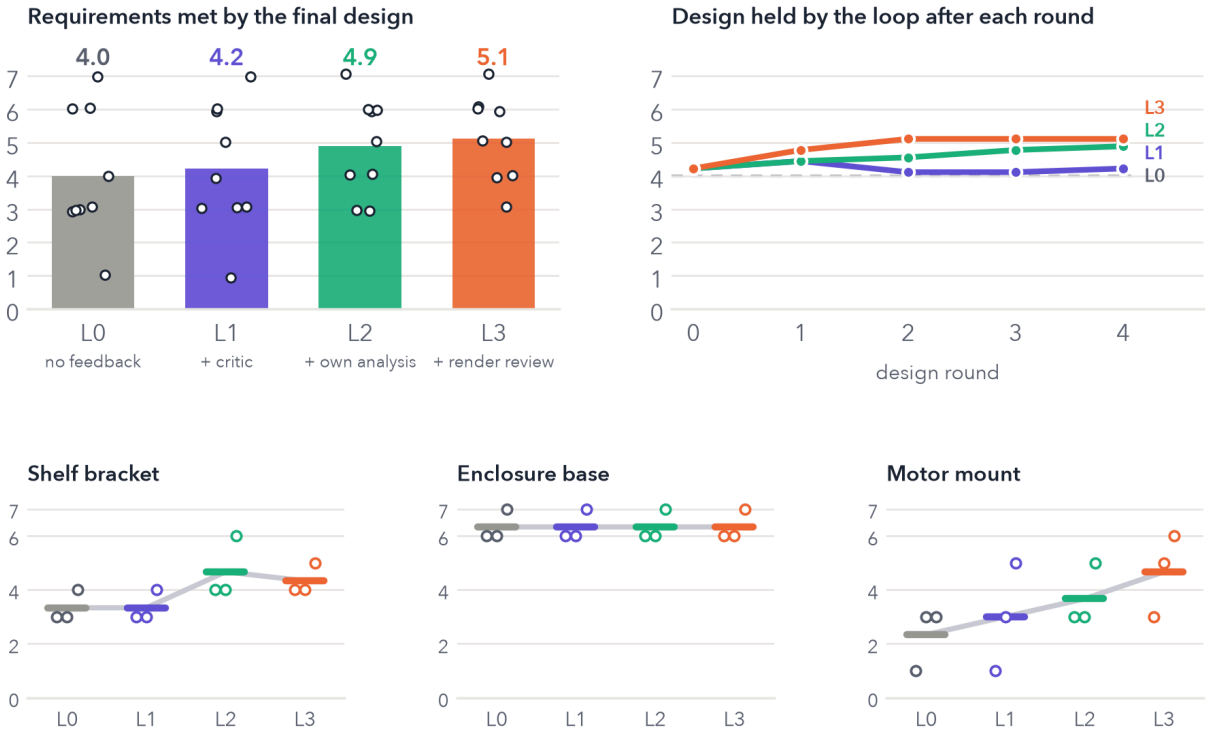


Fig. 3. Top left: requirements met by the final design; each dot is one run. Top right: the design the loop holds after each round, mean of nine runs (dashed: L0; the curves start above it because one first design was repaired further at the higher levels). Bottom: each part; dots are repeats, bars their mean.



Fig. 4. A correct reference design and the final design of each level, first repeat; "runs" lists all three repeats. **Shelf bracket:** two wall-screw holes a screwdriver can reach, a seat 110 mm up reaching 150 mm out, 150 N within stress and deflection limits, at most 90 g. **Enclosure base:** an 85 × 55 mm tray with a 2 mm floor, room for the board, four screw bosses, a USB-C cutout, at most 70 g. **Motor mount:** a 4 mm face plate with a 22.5 mm bore and four screw holes, clearance for the motor, a base with two bolt holes, at most 60 g.

4.2 Three critics, one design

Fig. 5 puts the three critics before the same first design: a motor mount whose face plate was built lying in the wrong plane, with its base below the table.



Fig. 5. The same first design under each level (first repeat): the part before, what the critic read and wrote, the change in the design document, the part after. Badges: requirements met. Quotes in all figures are verbatim.

The critic at L1 cannot know this. The document describes the plate correctly, so the critic looks for what the text leaves open, finds that the clearance pocket has no dimensions, and has them added. The document is better; the part is not.

At L2 the agent's probes come back empty where the bore and screw holes should be, and the critic reports exactly that. It is true, but it describes symptoms. The designer makes the plate a solid block, the part is still wrong, and four rounds pass without progress.

At L3 the reviewer looks at the views and says what a person would say: the plate is in the wrong plane and the base is below the table. One rewrite later the part meets six requirements. Numbers told the agent what was wrong. The picture told it why.

4.3 Round by round

When a number can name the fault, the numerical loop works as an engineer would hope (Fig. 6, top; Fig. 7). A first bracket weighs 445 g against a limit of 90 g. The critic reports the figure, the designer turns the solid section at the wall into a hollow one, and the next part weighs 43 g and meets six requirements. The next round goes for what is left, the screwdriver access, and does not get it.



Fig. 6. Two loops, round by round: each round's part, the requirements it meets, whether the loop kept it, and the first problem the critic named in the design before it. Framed: the final design.

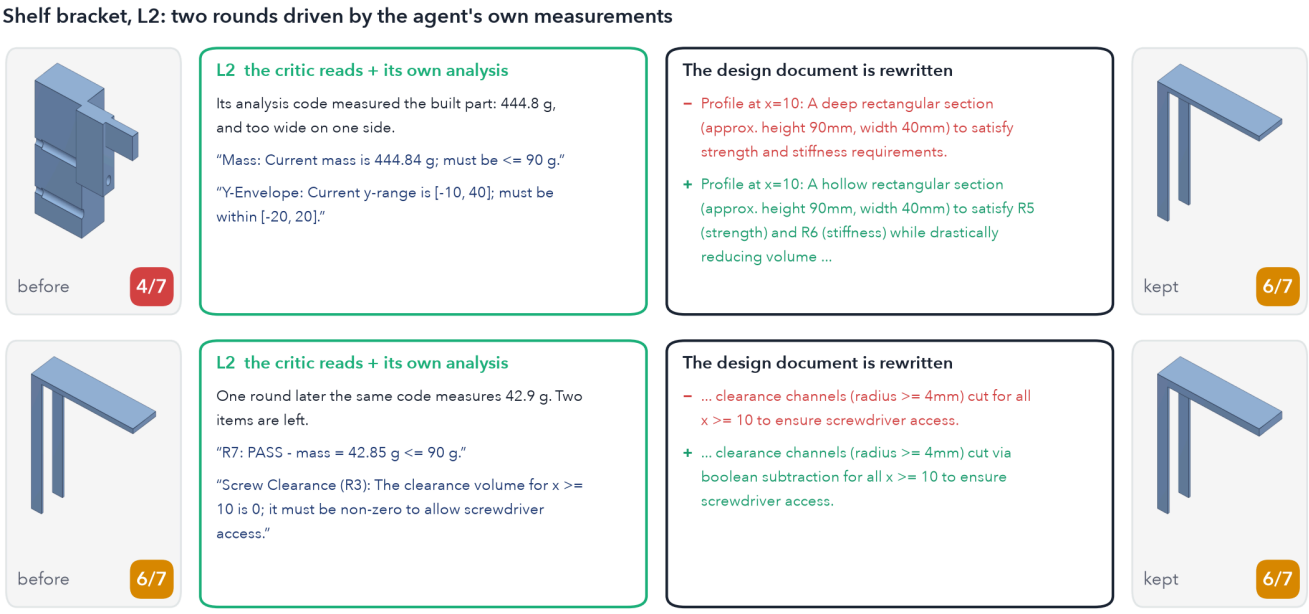


Fig. 7. The bracket's first two rounds in Fig. 6, in detail.

The motor mount in Fig. 6 (bottom) and Fig. 8 improves the same way, one fault at a time: first the material inside the motor's clearance space, then the bore. Its last round is the best part of all, and the loop does not keep it; Section 4.4 explains why.

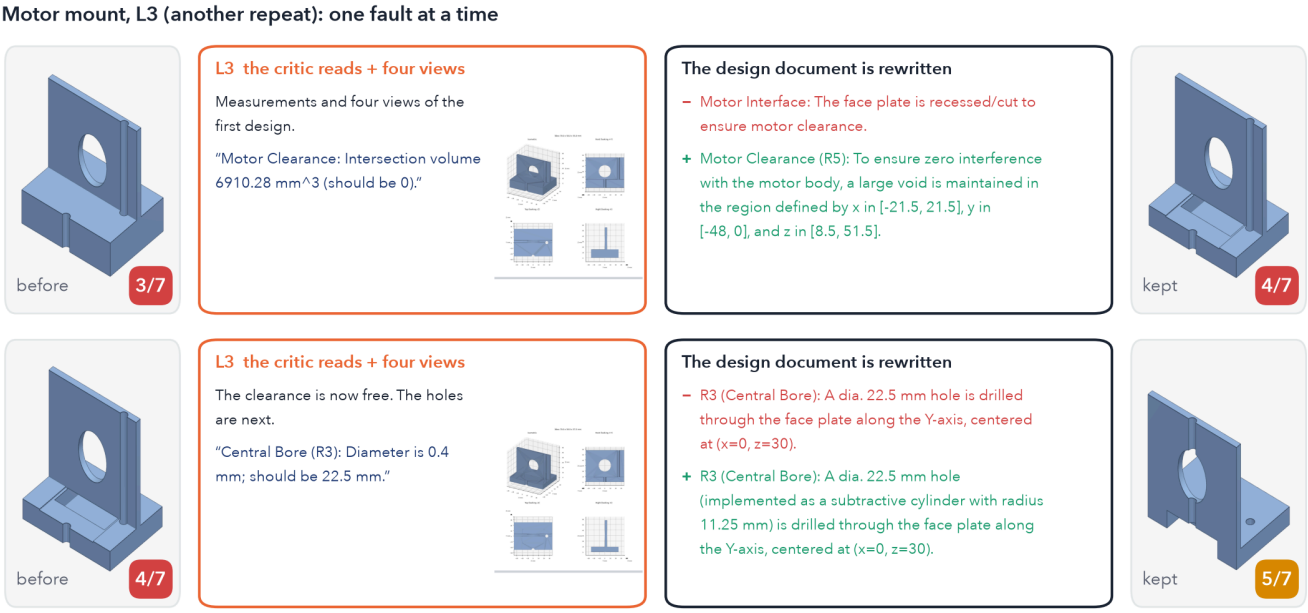


Fig. 8. The motor mount's first two rounds in Fig. 6, in detail.

4.4 What holds the loop back

Three things do (Fig. 9).

The first is repair. A design round fixes about one unmet requirement in seven at L1 and L2 and one in three at L3, and breaks roughly one in ten that were fine. Knowing the fault is not the same as removing it.

The second is the agent's own checks. After the control run they almost never miss a real fault: of 207 unmet requirements, they reported 201. But about one verdict in ten calls a met requirement unmet, for example because a probe reaches beyond its feature. At L2 and L3 the loop produced nine designs better than the one it held and did not keep them; eight of those carried a false alarm. This is how the best motor mount of Fig. 6 was dropped. An audit of every failed check removed about a third of the false alarms but let real faults through; by a rule set beforehand we dropped it.

The third is the eye itself. A quarter of the reviewer's remarks that name a requirement concern one the part already meets. The picture is the richest channel and the least reliable, which is why it works best beside measurements, not instead of them.

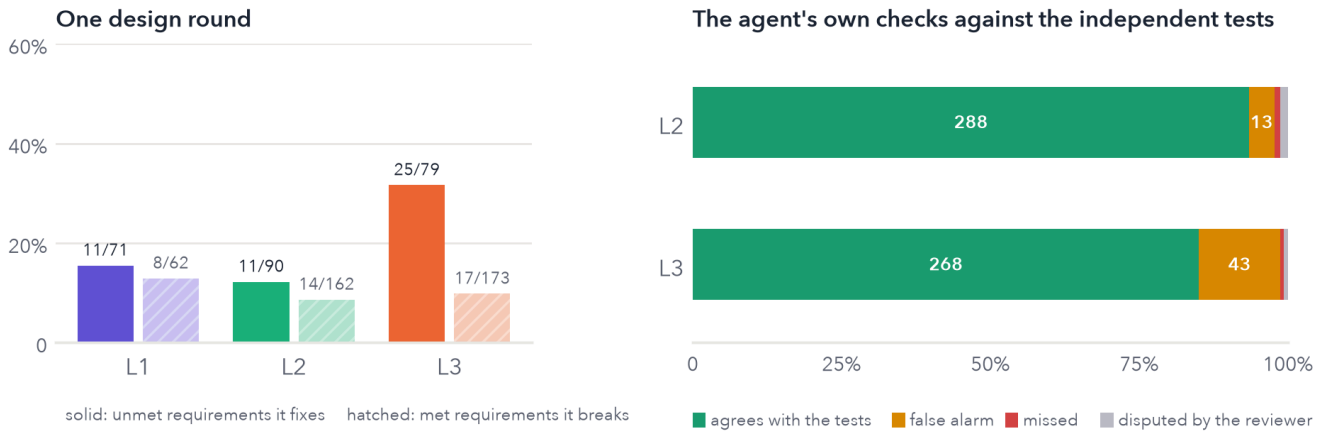


Fig. 9. Left: what one design round does. Right: the agent's own verdicts, compared with the independent tests.

4.5 Other parts

Four more parts were run twice each (Fig. 10): a phone stand, a pipe clamp, a drone arm and a heat sink. None was used for the revisions listed in Section 7. The pattern repeats: the designs meet 3.9 requirements at L0, 3.6 at

L1, 4.6 at L2 and 4.7 at L3; three runs improve from L0 to L3 and none gets worse. Single runs vary as before: one phone stand climbs from two requirements to five, and one heat sink reaches all six only at L3. Two runs are left out because the CAD agent ran out of repair steps in most of their builds, so their scores reflect its limit, not the feedback: a tilted sensor mount (12 of 16 builds; 0, 2, 2, 2 from L0 to L3) and the second pipe clamp (10 of 16; 2, 4, 3, 5).



Fig. 10. All seven parts: a correct reference, requirements met at each level (mean of the repeats), and the part L3 kept.

5 Discussion

How should a local model be used for design? The ladder suggests three answers. Let the model act, not only think: a critic that reads its own document improves the document, and the part barely follows. Let it measure, and make it test its measurements: analysis code written by the model is the cheapest reliable evidence it can have, and a control part that should fail everything exposes a check that measures nothing. And let it look: the rendered views hold the cause where the numbers hold only the symptom, although the eye raises false objections and should not judge alone. What remains hard is not seeing the fault but removing it without breaking something else, and deciding fairly whether the new part is better; the loop produced better designs than it kept. The next gains should come from the repair step, from the comparison of candidates, and from letting the agent try the part in use in a physics engine such as MuJoCo [15].

6 Related work

TextGrad [1] passes critiques backwards through text variables, and Self-Refine [2], Reflexion [3], Self-Debugging [4] and CodeT [5] improve outputs through self-feedback, execution or self-written tests, although self-critique without outside evidence is unreliable [6, 7]. For CAD, CADCodeVerify [8] repairs CadQuery code from questions about renders, Seek-CAD [9] and CADSmith [10] combine a model with render or kernel feedback, and Physics-

in-the-Loop [11] and Son et al. [12] add finite-element analysis through a fixed solver. Here a local open model writes its own analysis code, that code is itself tested on a control part, and the three kinds of feedback are added one at a time. As in SWE-bench [14], tests the agent cannot see decide the outcome.

7 Limitations

Claude wrote the problems and independent tests and assisted with the code; freezing the tests beforehand and checking them on reference and wrong designs reduces the risk of bias without removing it. Nine runs per level is a small sample, single runs vary a lot, and the further parts ran twice. The loop was revised after first passes over the three main parts exposed faults in it (a critic that argued with itself, checks that passed on missing geometry, a reviewer that discarded true measurements); every fix is general and all reported runs use the final version, but those parts were not unseen when it was fixed. The analysis is geometric and analytic, not finite-element.

8 Reproducibility

Everything runs offline:

```
python -m designgrad.run --ladder v6 --problems P01 --levels L0 L1 L2 L3 --out runs/x
python eval/v4_analyze.py runs/x --out runs/x
```

Every model call is recorded; replaying the records reproduces the runs we checked byte for byte. Code, problems, tests and records will be released.

Critic

```
Review the design. Write your answer in three parts with exactly
these headings.

## Working
Short notes, requirement by requirement: do your arithmetic and
weigh the evidence here. You may change your mind here.

## Verdicts
Exactly one line per requirement R1..Rn, in the form "Rk:
PASS|FAIL|UNSURE - reason" with the numbers you rely on. These are
final: one verdict per line, no second thoughts, nothing else in
this part.

## Problems
The problems to fix, most severe first. Be to the point: for each
problem name the feature, the value that is wrong and the value it
should have. No general advice. If nothing needs fixing, write
"None".
```

Reviewer of the agent's checks: the control run

```
The same script was also run on a control part: a 1 mm cube in
place of the real part, so almost nothing is there. On the control
part it reported:
- pass: R1, R4, R7
- fail or error: R2, R3, R5, R6
A 1 mm cube has no plate, wall, hole, bore, boss, slot or surface.
If a requirement asks for a feature to EXIST, or for material to
be somewhere, and its check still passes on the cube, then the
check only tests that a region is empty; it is NOT faithful. It
must also measure the material the feature belongs to (for a hole:
the opening is empty AND the material around it is solid).
Requirements that only set a limit (mass, envelope, a keep-out or
clearance zone, a single valid solid) are rightly met by the cube;
do not mark those unfaithful for this reason. Take the control run
into account in your JSON answer. This control rule concerns ONLY
the requirements listed under "pass" above. A check that fails or
errors on the cube has passed the control: judge it on the five
points alone. A check that reports FAIL on the real part is not
unfaithful merely because its value is surprising or contradicts
the design document: the part may really be wrong there.
```

Critic at L1: nothing but the design document

```
None beyond the design document. Do not take the document's own
claims on trust. In your working, recompute every number it claims
from the dimensions it states (mass from volume and density,
stresses, deflections, positions, clearances, sizes) and check
that the stated features and coordinates really satisfy each
requirement. If the document does not state the dimensions needed
to decide a requirement, the verdict is UNSURE and the missing
dimension is a problem to fix.
```

Analysis agent: two of its rules

```
- A hole, bore, slot or cutout is a feature IN material. Check
  both: the opening is empty AND the material it passes through is
  there (filled fraction of a ring or shell just outside the
  opening). An empty region alone does not show a hole: empty
  space has no holes.
- Compare measured values with limits using a small tolerance
  (0.01 mm for lengths, 0.1 % otherwise): CAD kernels return
  values such as 20.0000001 for 20.
```

Fig. 11. Four of the prompts, verbatim.

References

[1] Yuksekgonul et al. TextGrad. *Nature* 639, 2025.
[2] Madaan et al. Self-Refine. NeurIPS 2023.
[3] Shinn et al. Reflexion. NeurIPS 2023.
[4] Chen et al. Self-Debugging. ICLR 2024.
[5] Chen et al. CodeT. ICLR 2023.
[6] Huang et al. LLMs cannot self-correct reasoning yet. ICLR 2024.
[7] Olausson et al. Is self-repair a silver bullet for code generation? ICLR 2024.
[8] Alrashedy et al. CADCodeVerify. ICLR 2025.
[9] Li et al. Seek-CAD. ICLR 2026.
[10] Barkley et al. CADSmith. arXiv:2603.26512.
[11] Berger et al. Physics-in-the-Loop. IJCAI-ECAI 2026.
[12] Son et al. Self-improving CAD agents with FEA feedback. arXiv:2605.17448.
[13] Gemma Team. Gemma 4 technical report. arXiv:2607.02770.
[14] Jimenez et al. SWE-bench. ICLR 2024.
[15] Todorov et al. MuJoCo. IROS 2012.